

Project Report: LeapCard_Automation

Name: Madhava Lakshmi Satya Mohan Samanthakurthi

Student ID: x24123684

Module Name: Intelligent Agents and Process Automation (MSCAIJAN25I)

1. Introduction

The report elaborates on the design, development, and deployment of the RPA solution "LeapCard_Automation," which aims to automate the validation of student Leap Card applications. The project purposes to solve the inefficiencies and scope for human-induced errors in manually processing these applications.

Currently, the manual system can be described as repeating tasks, very slow processing, and inconsistent application of business rules. This automation wants to develop a strong and reliable process-facing high volumes of applications with speed and accuracy.

The project scope includes the following:

- Read applicant data from a structured file system
- Apply a series of business rules to each application for validation
- Make decisions to approve, put under manual review, or reject applications
- Generate consolidated output reports in CSV format
- Automatically inform applicants of their approval status via email

The solution is developed in UiPath Studio and is meant to execute on a local machine. All file operations use relative paths to maintain portability and provide offline use.

2. As-Is Process Analysis

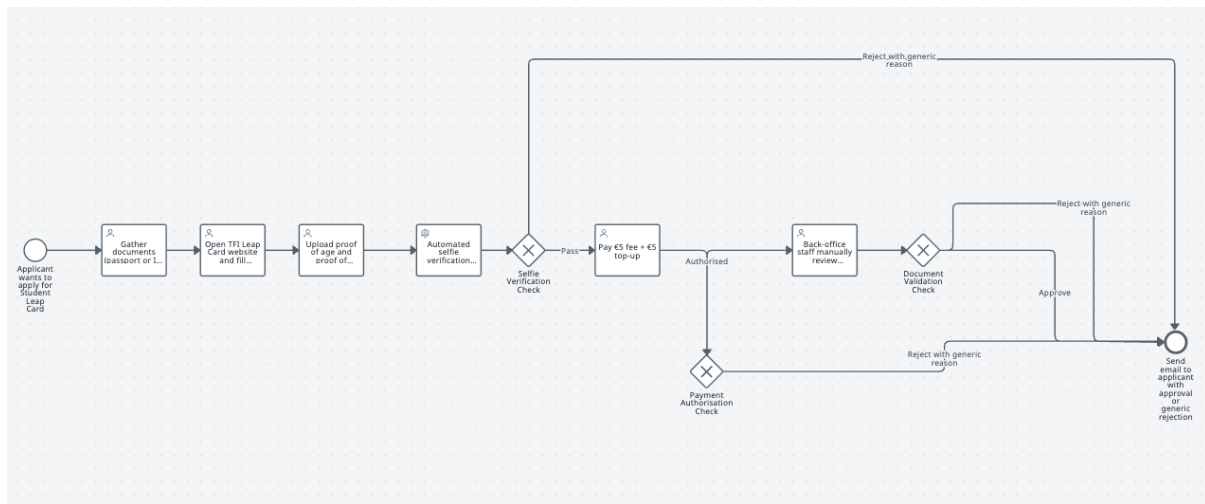
The manual process for validating Leap Card applications is a linear and labor-intensive workflow.

Process Description:

An administrative officer receives a batch of digital application folders. For each application, the officer must:

1. Open the applicant's folder.
2. Open the `applicant_data.csv` file to read the core data.
3. Manually calculate the applicant's age based on the date of birth provided.
4. Verify that the calculated age falls within the eligible range (17-25).
5. Check if the academic year is the current, valid year.
6. Confirm that the payment has been authorized.
7. Visually inspect the selfie and compare it to the ID document (passport) to estimate a match. This is highly subjective.
8. Based on these checks, decide the outcome: approved, rejected, or flagged for a second opinion (manual review).
9. Open the corresponding master spreadsheet (e.g., `approved_applications.xlsx`).

10. Copy and paste the applicant's details into the correct spreadsheet.
11. Manually compose and send an email to the applicant to inform them of the outcome.



Issues with the Manual Process:

- **Time-Consuming:** The process involves numerous manual steps, making it slow and inefficient, especially during peak application periods.
- **Prone to Error:** Manual data entry and validation are susceptible to human error, leading to incorrect rejections or approvals.
- **Subjectivity:** The selfie-to-ID match is subjective and can lead to inconsistent outcomes.
- **Lack of Auditability:** Tracking the status of an application and the reason for its outcome is difficult without a centralized, automated logging system.

3. To-Be Process Design

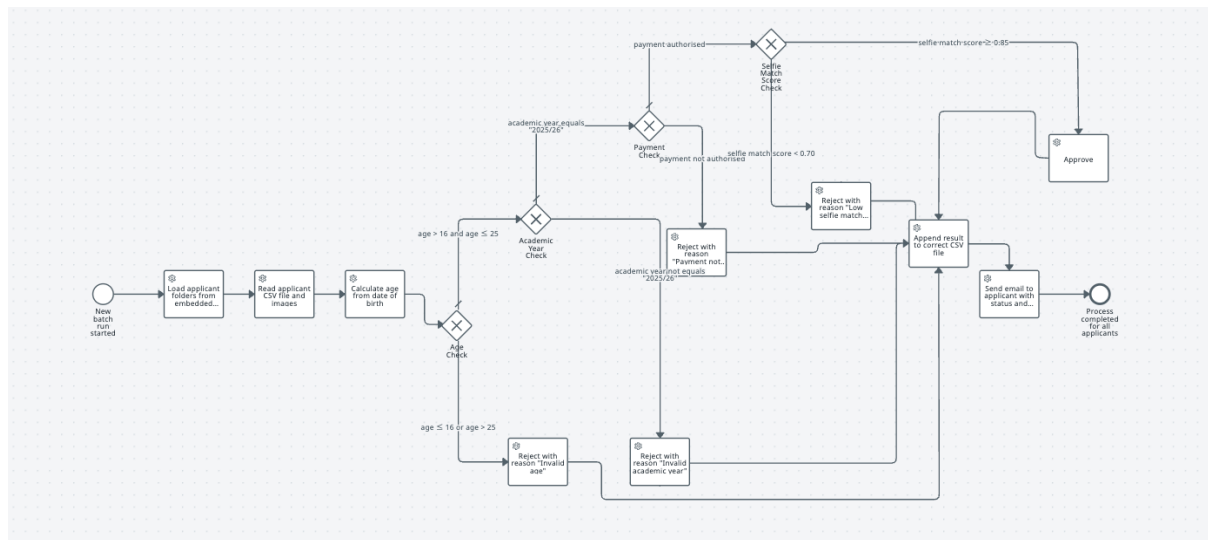
The "LeapCard_PoC" solution introduces a fully automated workflow that streamlines the validation process from start to finish.

Process Description:

The RPA bot initiates the process. It first prepares the environment by creating a clean output directory for the current run. The bot then systematically iterates through each applicant's subfolder within the main "SampleApplicants" directory. For each applicant, the bot performs the following actions:

1. Reads the data from the applicant_data.csv file.
2. Calculates the applicant's age and validates it against the required range (17-25).
3. Validates the academic year, payment authorization status, and the selfie match score against the predefined business rules.
4. Based on the validation results, it determines the application status: "Approved," "Manual Review," or "Rejected," along with the corresponding reason.
5. Writes the result (application ID, full name, status, and reason) to the appropriate CSV file (approved.csv, manual_review.csv, or rejected.csv).
6. Sends a formatted email notification to the applicant with the outcome and reason.

The process repeats until all applicant folders have been processed.



Benefits of Automation:

- **Speed and Efficiency:** The bot can process hundreds of applications in a fraction of the time required by a human user.
- **Accuracy and Consistency:** Business rules are applied uniformly to every application, eliminating human error and subjectivity.
- **Improved Auditability:** The output CSV files provide a clear, timestamped, and auditable record of all processed applications and their outcomes.
- **24/7 Operation:** The bot can be scheduled to run at any time, including outside of business hours, ensuring continuous processing.

4. RPA Tool and Technology Selection

The primary tool selected for this project is **UiPath Studio**. This choice is justified by UiPath's position as a market-leading RPA platform that offers a user-friendly, visual workflow designer, robust debugging tools, and a comprehensive library of built-in activities for handling files, data manipulation, and integrations.

Technologies Used:

- **UiPath Studio:** For designing and developing the core automation workflow.
- **UiPath Activities:**
 - o **File Activities:** (Path Exists, Delete Folder, Create Directory, Enumerate Directories, Read CSV, Write CSV) for managing the data and output files.
 - o **Data Manipulation:** Standard VB.NET functions within UiPath for date/time calculations (age) and conditional logic (If/Else If activities).
 - o **SMTP Mail:** The Send SMTP Mail Message activity for dispatching email notifications through a standard Gmail account, demonstrating integration with external services.
- **Data Sourcing:**
 - o **Kaggle:** The synthetic passport dataset from Kaggle was used as a basis for creating realistic applicant profiles.
(<https://www.kaggle.com/datasets/trainingdatapro/generated-passports-segmentation>).
 - o **ChatGPT:** Used to generate supplementary sample details to populate the applicant_data.csv files, ensuring a diverse set of test data.

5. Implementation and Development

The "LeapCard_PoC" workflow is contained within a single UiPath sequence that is heavily commented for clarity. The development follows the specific requirements outlined for the project.

Workflow Structure:

1. Initialization:

- o The OutputRootPath is defined using `Path.Combine(Environment.CurrentDirectory, "Output")`.
- o The workflow checks if this directory exists. If it does, all existing CSV files within it are deleted. If it doesn't exist, the directory is created.
- o Three new CSV files (`approved.csv`, `manual_review.csv`, `rejected.csv`) are created with the required headers: `application_id,full_name,status,reasons`.

2. Data Processing Loop:

- o The SampleRootPath is defined using `Path.Combine(Environment.CurrentDirectory, "SampleApplicants")`.
- o The workflow uses a For Each activity to loop through each subdirectory within SampleRootPath.

3. Business Rule Evaluation (Inside the Loop):

- o For each applicant folder, the `applicant_data.csv` is read into a DataTable.
- o **Age Calculation:** The date of birth is read, and the applicant's age is calculated as of the current date. An If condition checks if the age is between 17 and 25 (inclusive of 25). If not, the application is marked for rejection with the reason "Applicant age outside allowed range (17–25)".
- o **Academic Year:** An If condition checks if the `academic_year` column equals "2025/26". If not, the application is rejected with the reason "Invalid academic year".
- o **Payment Authorization:** An If condition checks if `payment_authorised` is True. If not, it's rejected with the reason "Payment not authorised".
- o **Selfie Match Score:** A nested series of If/Else If conditions evaluates the `live_selfie_match_score`:
 - If < 0.70 , it is rejected with the reason "Low selfie match score (< 0.70)".
 - If ≥ 0.70 and < 0.85 , it is flagged for "Manual Review" with the reason "Selfie match score in manual review band (0.70–0.85)".
 - If ≥ 0.85 and all previous checks have passed, the status is set to "Approved".

4. Output and Notification:

- o A Switch activity is used based on the final status ("Approved", "Manual Review", "Rejected").
- o The corresponding case appends the applicant's data (`application_id`, `full_name`, `status`, `reasons`) to the correct output CSV file.
- o The Send SMTP Mail Message activity is then called. It is configured with input arguments for server details and credentials. The subject and body of the email are dynamically populated with the applicant's name, application ID, and the outcome of the validation.

6. Testing and Results

A structured testing plan was executed to ensure the reliability and accuracy of the automation.

Test Plan:

Test cases were designed to cover every business rule and decision branch:

- **Approved:** An applicant profile meeting all criteria (age 20, correct academic year, payment true, selfie score 0.90).
- **Rejected (Age):** An applicant aged 16 and another aged 26.
- **Rejected (Academic Year):** An applicant with academic year "2024/25".
- **Rejected (Payment):** An applicant with `payment_authorised` set to `False`.
- **Rejected (Selfie):** An applicant with a selfie score of 0.65.
- **Manual Review:** An applicant with a selfie score of 0.75.
- **Multiple Rejection Reasons:** An applicant who is too young and has an invalid academic year.

Summary of Results:

The bot performed flawlessly across all test cases. In each scenario, the applicant's data was written to the correct output CSV file with the appropriate status and reason.

- The "Approved" applicant appeared only in `approved.csv`.
- All "Rejected" applicants were correctly logged in `rejected.csv` with the first reason for failure encountered by the logic.
- The "Manual Review" applicant was correctly written to `manual_review.csv`.
- Email notifications were successfully generated and sent for each case with the correct dynamic content in the subject and body.

The output files served as a clean and accurate log of the bot's run, confirming that the data handling and business logic were implemented correctly.

7. Cost-Benefit Analysis

This analysis provides a high-level estimate of the costs and benefits associated with deploying the "LeapCard_PoC" solution.

Assumptions:

- **Manual Processing Time:** 5 minutes per application.
- **Application Volume:** 10,000 applications per year (concentrated in late summer/early autumn).
- **Administrative Salary:** Average salary for an Administrative Assistant in Dublin is approximately €32,000 per year, or €16.41 per hour (based on a 37.5-hour week).

Costs:

- **Development Cost:**
 - Estimated development and testing time: 20 hours.
 - Assuming a freelance developer rate of €50/hour, the one-time development cost is **€1,000**.
- **Software Cost:**
 - UiPath Community Edition was used, so software licensing cost is €0 for this PoC.
- **Operational Cost:**
 - The bot runs on existing infrastructure, so operational costs are negligible.

Benefits (Annual Savings):

- **Time Saved:**
 - Total manual processing time: 10,000 applications * 5 minutes/application = 50,000 minutes = 833.3 hours.

- **Cost Savings:**
 - o Annual cost of manual processing: 833.3 hours * €16.41/hour = **€13,674**.
- **Return on Investment (ROI):**
 - o $ROI = (Annual\ Savings - Annual\ Cost) / Total\ Investment$
 - o $ROI = (€13,674 - €0) / €1,000 = \mathbf{1367\%}$ in the first year.

In addition to direct cost savings, the automation provides significant intangible benefits, including higher accuracy, better compliance with business rules, and improved applicant satisfaction due to faster processing times.

8. Conclusion

The "LeapCard_PoC" project successfully demonstrates the power of Robotic Process Automation in transforming administrative workflows. The developed solution effectively automates the end-to-end process of Leap Card application validation, adhering to all specified business rules and requirements. The bot is efficient, accurate, and provides a full audit trail through its structured output files and email notifications.

Future Improvements:

- **OCR Integration:** Enhance the bot to use Optical Character Recognition (OCR) to extract data directly from scanned documents like passports and student ID cards, removing the dependency on a pre-populated CSV file.
- **AI-Powered Facial Recognition:** Integrate a dedicated AI service, such as the Microsoft Azure Face API, to perform a more accurate and reliable comparison between the selfie and the ID photo, replacing the sample match score.
- **Exception Handling and Unattended Deployment:** Implement a more robust exception handling framework and deploy the bot in an unattended mode on a server, allowing it to run on a schedule without human intervention.

This project serves as a strong foundation for a production-grade automation solution that can deliver significant value to any organization managing similar application processes.

9. References

- Kaggle. (2020). *Generated Passports Segmentation*. Available at: <https://www.kaggle.com/datasets/trainingdatapro/generated-passports-segmentation> (Accessed: 8 August 2025).
- Leap Card. (2024). *Young Adult and Student Card Information*. Available at: <https://about.leapcard.ie/young-adult-and-student-card-launch> (Accessed: 8 August 2025).
- Lucidchart. (n.d.). *BPMN Tutorial*. Available at: <https://www.lucidchart.com/pages/tutorial/bpmn> (Accessed: 8 August 2025).
- UiPath. (n.d.). *Send Mail Activities*. Available at: <https://docs.uipath.com/activities/other/latest/productivity/send-mail> (Accessed: 8 August 2025).
- UiPath Marketplace. (n.d.). *Microsoft Facial Recognition*. Available at: <https://marketplace.uipath.com/listings/microsoft-facial-recognition> (Accessed: 8 August 2025).